

Практичне заняття №6

Interceptors + Exception Filters + Swagger

Дисципліна: Високорівневі мови програмування та фреймворки

Мета

Перетворити MiniShop API з навчального проєкту на production-ready сервіс: додати `LoggingInterceptor` для моніторингу часу виконання запитів, `TransformInterceptor` для єдиного формату відповідей, `HttpExceptionHandler` для стандартизованої обробки помилок з `traceld`, та `Swagger/OpenAPI` документацію з інтерактивним UI для тестування API.

Передумови

Виконане Практичне заняття №5: MiniShop API з автентифікацією (JWT), авторизацією (Guards, RBAC), DTO з валідацією. Всі 10+ ендпоінтів працюють через `docker compose`.

Що здаємо (тільки одне)

Лінк на GitHub репозиторій, в якому є:

- README.md з коротким звітом (шаблон нижче) + скріншот Swagger UI
- `LoggingInterceptor` та `TransformInterceptor` у `src/common/interceptors/`
- `HttpExceptionHandler` у `src/common/filters/`
- Swagger документація доступна на `/api/docs`
- Декоратори Swagger на DTO та контролерах
- Мінімум 3 нових коміти

Важливо: Swagger UI має відкриватись на `http://localhost:3000/api/docs` і показувати всі ендпоінти з описами, параметрами та прикладами відповідей.

Вправа 1 — Створити `LoggingInterceptor`

Опис

Створити `Interceptor`, який логує метод, URL та час виконання кожного запиту. `Interceptor` обгортає виконання контролера: він має доступ і до «до», і до «після». В логах NestJS з'являться рядки типу: `"GET /api/products — 12ms — 200"`.

Інструкції (покроково)

1) Створити файл `src/common/interceptors/logging.interceptor.ts`:

```
mkdir -p src/common/interceptors
```

```
import {
  Injectable,
  NestInterceptor,
  ExecutionContext,
  CallHandler,
  Logger,
} from '@nestjs/common';
import { Observable } from 'rxjs';
import { tap } from 'rxjs/operators';

@Injectable()
export class LoggingInterceptor
  implements NestInterceptor
{
  private readonly logger = new Logger('HTTP');

  intercept(
    context: ExecutionContext,
    next: CallHandler,
  ): Observable<any> {
    const req = context.switchToHttp().getRequest();
    const { method, url } = req;
    const now = Date.now();

    return next.handle().pipe(
      tap(() => {
        const res = context
          .switchToHttp()
          .getResponse();
        const ms = Date.now() - now;
        this.logger.log(
          `${method} ${url} - ${res.statusCode} - ${ms}ms`,
        );
      }),
    );
  }
}
```

Пояснення

Елемент	Що робить
NestInterceptor	Інтерфейс з методом intercept(context, next). next.handle() — виклик контролера.
next.handle()	Повертає Observable — потік відповіді контролера. Без виклику handle() контролер не виконається.
tap()	Оператор RxJS: виконує побічну дію (логування) без зміни відповіді. Спрацьовує ПІСЛЯ контролера.
Date.now()	Засікаємо час ДО контролера. В tap() обчислюємо різницю — отримуємо час виконання.
Logger('HTTP')	Вбудований NestJS logger з префіксом [HTTP]. Виводить у stdout з кольорами та timestamp.

2) Зареєструвати глобально в main.ts:

```
import { LoggingInterceptor }
  from './common/interceptors/logging.interceptor';

// Додати ПЕРЕД app.listen:
app.useGlobalInterceptors(new LoggingInterceptor());
```

3) Перезапустити та перевірити:

```
docker compose restart app
curl http://localhost:3000/api/products
docker compose logs --tail=5 app
```

Очікується у логах:

```
[HTTP] GET /api/products - 200 - 12ms
```

Очікуваний результат

- Кожен HTTP-запит логується з методом, URL, статус-кодом та часом виконання
- Логи видно в docker compose logs

Вправа 2 — Створити TransformInterceptor

Опис

Створити `Interceptor`, який обгортає всі успішні відповіді у стандартну структуру: `{ data: ..., statusCode: 200, timestamp: "..."}` . Це забезпечує єдиний формат для фронтенду — клієнт завжди знає, де шукати дані.

Інструкції (покроково)

1) Створити файл `src/common/interceptors/transform.interceptor.ts`:

```
import {
  Injectable,
  NestInterceptor,
  ExecutionContext,
  CallHandler,
} from '@nestjs/common';
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';

export interface Response<T> {
  data: T;
  statusCode: number;
  timestamp: string;
}

@Injectable()
export class TransformInterceptor<T>
  implements NestInterceptor<T, Response<T>>
{
  intercept(
    context: ExecutionContext,
    next: CallHandler,
  ): Observable<Response<T>> {
    const statusCode = context
      .switchToHttp()
      .getResponse().statusCode;

    return next.handle().pipe(
      map((data) => ({
        data,
        statusCode,
        timestamp: new Date().toISOString(),
      })),
    );
  }
}
```

Різниця між `tap()` і `map()`: `tap()` виконує побічну дію (логування) без зміни даних. `map()` трансформує дані — саме те, що потрібно для обгортання відповіді.

2) Зареєструвати глобально в `main.ts`:

```
import { TransformInterceptor }
  from './common/interceptors/transform.interceptor';

app.useGlobalInterceptors(
  new LoggingInterceptor(),
  new TransformInterceptor(),
);
```

3) Перезапустити та перевірити:

```
docker compose restart app
curl http://localhost:3000/api/products
```

Очікується:

```
{
  "data": [ ... масив продуктів ... ],
  "statusCode": 200,
  "timestamp": "2025-01-15T10:30:00.000Z"
}
```

💡 Тепер **КОЖНА** відповідь обгорнута у `{ data, statusCode, timestamp }`.
Фронтенд-розробник завжди знає: дані — в полі `data`, помилки — в іншому форматі (`Exception Filter`). Це контракт.

Очікуваний результат

- Всі успішні відповіді мають формат `{ data, statusCode, timestamp }`
- Дані ендпоінтів знаходяться всередині поля `data`

Вправа 3 — Створити `HttpExceptionFilter`

Опис

Створити глобальний `Exception Filter`, який перехоплює всі HTTP-помилки і формує єдиний формат з полями `error.code`, `error.message`, `error.details` та `traceld`. Це те, про що ми говорили на Лекції 3: стандартизований формат помилок для фронтенду.

Інструкції (покроково)

1) Створити файл `src/common/filters/http-exception.filter.ts`:

```
mkdir -p src/common/filters
```

```
import {
  ExceptionFilter,
  Catch,
  ArgumentsHost,
  HttpException,
  HttpStatus,
  Logger,
} from '@nestjs/common';
import { randomUUID } from 'crypto';
import { Request, Response } from 'express';

@Catch()
export class HttpExceptionFilter
  implements ExceptionFilter
{
  private readonly logger = new Logger('Exception');

  catch(exception: unknown, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const request = ctx.getRequest<Request>();
    const response = ctx.getResponse<Response>();
    const traceId = randomUUID();

    let status = HttpStatus.INTERNAL_SERVER_ERROR;
    let message = 'Internal server error';
    let details: any = undefined;

    if (exception instanceof HttpException) {
      status = exception.getStatus();
      const body = exception.getResponse();
      if (typeof body === 'string') {
        message = body;
      } else if (typeof body === 'object') {
        const obj = body as any;
        message = obj.message || obj.error || message;
        // ValidationPipe повертає масив помилок
        if (Array.isArray(obj.message)) {
          details = obj.message;
          message = 'Validation failed';
        }
      }
    }
  }
}
```

```

        // Логуюємо з traceId для дебагу
        this.logger.error(
            `[${traceId}] ${request.method} ${request.url}` +
            ` - ${status} - ${message}`,
            exception instanceof Error
            ? exception.stack
            : undefined,
        );

        response.status(status).json({
            error: {
                code: status,
                message,
                ...(details && { details }),
                traceId,
            },
            timestamp: new Date().toISOString(),
        });
    }
}

```

Формат відповідей

Помилка валідації (400):

```

{
  "error": {
    "code": 400,
    "message": "Validation failed",
    "details": [
      "name must be longer than or equal to 2 characters",
      "price must not be less than 0.01"
    ],
    "traceId": "a1b2c3d4-e5f6-..."
  },
  "timestamp": "2025-01-15T10:30:00.000Z"
}

```

Помилка авторизації (401):

```

{
  "error": {
    "code": 401,
    "message": "Invalid or expired token",
    "traceId": "x7y8z9..."
  },
  "timestamp": "2025-01-15T10:31:00.000Z"
}

```

```
}
```

💡 **@Catch()** без параметрів перехоплює ВСІ виключення, не тільки *HttpException*. Якщо в сервісі виникне *TypeError* або помилка бази — клієнт отримає чисту 500 з *traceld*, а деталі будуть у логах.

2) Зареєструвати глобально в *main.ts*:

```
import { HttpExceptionFilter }
  from './common/filters/http-exception.filter';

// Додати ПЕРЕД app.listen:
app.useGlobalFilters(new HttpExceptionFilter());
```

3) Перезапустити та перевірити помилки:

```
docker compose restart app
```

```
# 404
curl http://localhost:3000/api/products/999

# 400 (валідація)
curl -X POST http://localhost:3000/api/categories \
  -H "Content-Type: application/json" \
  -d '{"name": ""}'
```

Очікується: помилки у новому форматі з *traceld*. Перевірте логи — той самий *traceld* має бути в *docker compose logs*.

Очікуваний результат

- Всі помилки мають формат { error: { code, message, traceld }, timestamp }
- *ValidationPipe* помилки включають details з масивом полів
- *traceld* у відповіді збігається з *traceld* у логах — для дебагу

Вправа 4 — Встановити та налаштувати Swagger

Опис

Встановити *@nestjs/swagger* та налаштувати автоматичну генерацію OpenAPI-документації. Swagger UI — це інтерактивна вебсторінка, де можна побачити всі ендпоінти API, їх параметри, формати запитів і відповідей, та протестувати запити прямо з браузера.

Інструкції (покроково)

1) Встановити пакет:

```
docker compose run --rm app npm install @nestjs/swagger
```

2) Налаштувати Swagger в main.ts — додати ПЕРЕД app.listen():

```
import { DocumentBuilder, SwaggerModule }
  from '@nestjs/swagger';

// Swagger setup
const config = new DocumentBuilder()
  .setTitle('MiniShop API')
  .setDescription(
    'REST API для навчального інтернет-магазину. ' +
    'Автентифікація через JWT Bearer token.',
  )
  .setVersion('1.0')
  .addBearerAuth()
  .build();

const document = SwaggerModule.createDocument(
  app,
  config,
);
SwaggerModule.setup('api/docs', app, document);
```

Пояснення: `addBearerAuth()` додає кнопку "Authorize" у Swagger UI для введення JWT-токена. `SwaggerModule.setup('api/docs', ...)` реєструє Swagger UI на URL `/api/docs`.

3) Перезапустити та відкрити Swagger:

```
docker compose restart app
```

Відкрити у браузері: <http://localhost:3000/api/docs>

Очікується: Swagger UI з усіма ендпоінтами, згрупованими за контролерами. Поки без описів — їх додамо в наступних вправах.

Очікуваний результат

- <http://localhost:3000/api/docs> відкриває Swagger UI
- Всі ендпоінти видні (auth, categories, products)
- Є кнопка Authorize для JWT

Вправа 5 — Додати Swagger-декоратори до DTO

Опис

Додати `@ApiProperty()` до кожного поля DTO — це дозволить Swagger показувати структуру запитів з описами, типами та прикладами значень. Без `@ApiProperty()` Swagger бачить тільки назви полів, але не знає типів і обмежень.

Інструкції (покроково)

1) Увімкнути Swagger plugin у nest-cli.json (автоматичне зчитування типів):

```
{
  "collection": "@nestjs/schematics",
  "sourceRoot": "src",
  "compilerOptions": {
    "plugins": ["@nestjs/swagger"]
  }
}
```

 *Swagger plugin автоматично зчитує типи TypeScript і декоратори class-validator (@IsString, @Min тощо). З увімкненим plugin вам потрібно менше ручних @ApiProperty(). Але ми все одно додамо їх для прикладів та описів.*

2) Оновити src/products/dto/create-product.dto.ts — додати @ApiProperty():

```
import { ApiProperty, ApiPropertyOptional }
  from '@nestjs/swagger';
import {
  IsString, IsNumber, IsOptional,
  IsInt, Min, MaxLength, MinLength,
} from 'class-validator';
```

```
export class CreateProductDto {
  @ApiProperty({
    example: 'iPhone 16',
    description: 'Назва продукту',
    maxLength: 255,
  })
  @IsString()
  @MinLength(2)
  @MaxLength(255)
  name: string;

  @ApiProperty({
    example: 999.99,
    description: 'Ціна у гривнях',
    minimum: 0.01,
  })
  @IsNumber({ maxDecimalPlaces: 2 })
  @Min(0.01)
```

```

price: number;

@ApiPropertyOptional({
  example: 'Flagship smartphone',
  description: 'Опис продукту',
})
@IsOptional()
@IsString()
@MaxLength(1000)
description?: string;

@ApiPropertyOptional({
  example: 50,
  description: 'Кількість на складі',
  default: 0,
})
@IsOptional()
@IsInt()
@Min(0)
stock?: number;

@ApiPropertyOptional({
  example: 1,
  description: 'ID категорії',
})
@IsOptional()
@IsInt()
categoryId?: number;
}

```

@ApiProperty vs @ApiPropertyOptional: @ApiProperty — обов'язкове поле (зірочка у Swagger). @ApiPropertyOptional — необов'язкове. Ці декоратори НЕ замінюють class-validator — вони тільки для документації.

3) Аналогічно додати @ApiProperty() / @ApiPropertyOptional() до:

- src/categories/dto/create-category.dto.ts
- src/auth/dto/register.dto.ts
- src/auth/dto/login.dto.ts

4) Перезібрати та перевірити:

```
docker compose up --build -d
```

Відкрити <http://localhost:3000/api/docs> — тепер DTO показують приклади, типи та описи полів.

⚠ **Після зміни `nest-cli.json` потрібно повний `rebuild`: `docker compose up --build`. Просто `restart` недостатньо — `plugin` працює на етапі компіляції.**

Очікуваний результат

- Swagger UI показує структуру DTO з типами, прикладами та описами
- Обов'язкові поля позначені зірочкою
- Натиснувши "Try it out" можна побачити приклад JSON-тіла

Вправа 6 — Додати Swagger-декоратори до контролерів

Опис

Додати декоратори `@ApiTags()`, `@ApiOperation()`, `@ApiResponse()` та `@ApiBearerAuth()` до контролерів. Це додає описи ендпоінтів, можливі відповіді та позначає, які ендпоінти потребують авторизації.

Інструкції (покроково)

1) Оновити `src/products/products.controller.ts`:

```
import {
  ApiTags, ApiOperation, ApiResponse,
  ApiBearerAuth,
} from '@nestjs/swagger';

@ApiTags('Products')
@Controller('api/products')
export class ProductsController {
  // ... constructor ...

  @Get()
  @ApiOperation({
    summary: 'Отримати всі продукти',
    description: 'Повертає список усіх продуктів з вкладеними категоріями. Публічний ендпоінт.',
  })
  @ApiResponse({ status: 200, description: 'Список продуктів' })
  findAll() { ... }

  @Get('/:id')
  @ApiOperation({ summary: 'Отримати продукт за ID' })
  @ApiResponse({ status: 200, description: 'Продукт знайдено' })
  @ApiResponse({ status: 404, description: 'Продукт не знайдено' })
}
```

```

findOne(...) { ... }

@Post()
@ApiBearerAuth()
@ApiOperation({ summary: 'Створити продукт (admin)' })
@ApiResponse({ status: 201, description: 'Продукт створено' })
@ApiResponse({ status: 400, description: 'Помилка валідації' })
@ApiResponse({ status: 401, description: 'Не авторизовано' })
@ApiResponse({ status: 403, description: 'Недостатньо прав' })
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles(Role.ADMIN)
create(@Body() dto: CreateProductDto) { ... }

@Patch('/:id')
@ApiBearerAuth()
@ApiOperation({ summary: 'Оновити продукт (admin)' })
@ApiResponse({ status: 200, description: 'Продукт оновлено' })
@ApiResponse({ status: 404, description: 'Продукт не знайдено' })
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles(Role.ADMIN)
update(...) { ... }

@Delete('/:id')
@ApiBearerAuth()
@ApiOperation({ summary: 'Видалити продукт (admin)' })
@ApiResponse({ status: 200, description: 'Продукт видалено' })
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles(Role.ADMIN)
remove(...) { ... }
}

```

Декоратор	Що робить
@ApiTags('Products')	Групує ендпоінти у Swagger UI під заголовком "Products"
@ApiOperation()	Додає summary та description до конкретного ендпоінта
@ApiResponse()	Описує можливі HTTP-відповіді з їх статус-кодами
@ApiBearerAuth()	Позначає ендпоінт як такий, що потребує JWT-токен. В Swagger з'являється замочок

2) Аналогічно додати декоратори до:

- `src/categories/categories.controller.ts` — `@ApiTags('Categories')`, `@ApiBearerAuth()` на `POST/PATCH/DELETE`
- `src/auth/auth.controller.ts` — `@ApiTags('Auth')`, `@ApiOperation()` на `register` та `login`

3) Перезібрати та перевірити:

```
docker compose up --build -d
```

Відкрити `http://localhost:3000/api/docs` — тепер ендпоінти мають описи, згруповані за тегами, а захищені — з замочком.

Очікуваний результат

- Swagger UI показує три групи: `Auth`, `Categories`, `Products`
- Кожен ендпоінт має `summary` та `description`
- Захищені ендпоінти позначені замочком (`ApiBearerAuth`)
- Можливі відповіді (200, 400, 401, 403, 404) описані

Вправа 7 — Тестування API через Swagger UI

Опис

Протестувати повний flow через Swagger UI замість `curl`: зареєструватись, залогінитись, авторизуватись через кнопку `Authorize`, створити продукт. Це демонструє, що Swagger — не тільки документація, а інтерактивний інструмент тестування.

Інструкції (покроково)

1) Відкрити `http://localhost:3000/api/docs` у браузері.

2) Знайти `POST /auth/login` у секції `Auth`. Натиснути "Try it out". Ввести JSON:

```
{
  "email": "admin@test.com",
  "password": "password123"
}
```

Натиснути "Execute". Скопіювати `accessToken` з відповіді.

3) Натиснути кнопку "Authorize" (зелений замочок у верхній частині сторінки). Вставити токен у поле `Value`. Натиснути "Authorize".

4) Знайти POST /api/products у секції Products. Натиснути "Try it out". Ввести JSON:

```
{
  "name": "MacBook Air M4",
  "price": 1299.99,
  "stock": 25
}
```

Натиснути "Execute". Очікується 201 Created з обгорнутою відповіддю { data: {...}, statusCode: 201, timestamp: "..."}.

5) Спробувати POST /api/products з порожнім name:

```
{
  "name": "",
  "price": -5
}
```

Очікується: помилка у форматі Exception Filter з details та traceId.

6) Перевірити логи — кожен запит з Swagger має з'явитись у LoggingInterceptor:

```
docker compose logs --tail=10 app
```

 *Swagger UI — це ваш вбудований Postman. На реальних проєктах нова людина в команді відкриває /api/docs і може почати працювати з API без жодних пояснень. Якщо ваш Swagger повний і точний — він замінює 90% документації.*

Очікуваний результат

- Повний auth flow працює через Swagger UI
- Успішні відповіді обгорнуті TransformInterceptor
- Помилки мають формат HttpExceptionHandler
- Логи показують запити від Swagger

Вправа 8 — Оновити README та зробити коміти

Інструкції (покроково)

1) Зробити скріншот Swagger UI (<http://localhost:3000/api/docs>) і зберегти як swagger-screenshot.png у корінь репозиторію.

2) Зробити коміти (мінімум 3):

```
git add src/common/interceptors/ src/common/filters/ src/main.ts
git commit -m "add LoggingInterceptor, TransformInterceptor, HttpExceptionHandler"
```

```

git add package.json nest-cli.json src/products/dto/ src/categories/dto/
src/auth/dto/
git commit -m "add swagger with ApiProperty decorators on DTOs"

git add src/products/products.controller.ts src/categories/categories.controller.ts
git add src/auth/auth.controller.ts
git commit -m "add swagger decorators to controllers"

git add README.md swagger-screenshot.png
git commit -m "update readme for practical 6, add swagger screenshot"

git push

```

3) Оновити README.md за шаблоном:

```

## Student
- Name: <ПІБ>
- Group: <Група>

```

```

## Практичне заняття №6 – Interceptors + Exception Filters + Swagger

```

```

### Структура репозиторію
...

```

```

.
├─ src/
│   ├─ auth/ ...
│   ├─ users/ ...
│   ├─ categories/ ...
│   ├─ products/ ...
│   ├─ common/
│   │   ├─ enums/
│   │   │   └─ role.enum.ts
│   │   ├─ guards/
│   │   │   ├─ jwt-auth.guard.ts
│   │   │   └─ roles.guard.ts
│   │   ├─ decorators/
│   │   │   ├─ current-user.decorator.ts
│   │   │   └─ roles.decorator.ts
│   │   └─ interceptors/
│   │       ├─ logging.interceptor.ts
│   │       └─ transform.interceptor.ts
│   │   └─ filters/
│   │       └─ http-exception.filter.ts
│   │   └─ pipes/
│   │       └─ trim.pipe.ts
│   └─ migrations/

```



```
|   |─ main.ts
|   └─ app.module.ts
|─ swagger-screenshot.png
|─ Dockerfile
|─ docker-compose.yml
└─ README.md
```
```

### Запуск проекту

```
```bash
cp .env.example .env
docker compose up --build
```
```

### Swagger UI

<http://localhost:3000/api/docs>

![Swagger] (swagger-screenshot.png)

### Формат успішної відповіді

```
```json
{
  "data": { ... },
  "statusCode": 200,
  "timestamp": "2025-01-15T10:30:00.000Z"
}
```
```

### Формат помилки

```
```json
{
  "error": {
    "code": 400,
    "message": "Validation failed",
    "details": ["name must be longer..."],
    "traceId": "a1b2c3..."
  },
  "timestamp": "2025-01-15T10:31:00.000Z"
}
```
```

### Приклад логів (LoggingInterceptor)

```
```text
<вивід docker compose logs з рядками [HTTP] GET /api/products ...>
```
```

```
Тест помилки з traceId
```text
<вивід curl GET /api/products/999>
```
```

## Troubleshooting — типові проблеми

### 1) Swagger UI порожній — немає ендпоінтів

Симптоми: /api/docs відкривається, але список ендпоінтів порожній.

#### Що зробити:

- Перевірити, що SwaggerModule.setup() викликається ПІСЛЯ створення документа
- Перевірити, що всі модулі (AuthModule, ProductsModule, CategoriesModule) імпортовані в AppModule
- Перезібрати повністю: docker compose up --build

### 2) @ApiProperty() не відображається у Swagger

Симптоми: DTO показують тільки назви полів без типів та описів.

#### Що зробити:

- Перевірити, що @nestjs/swagger встановлений: npm ls @nestjs/swagger
- Перевірити nest-cli.json — чи є plugin @nestjs/swagger у compilerOptions
- Обов'язково rebuild: docker compose up --build (plugin працює при компіляції)

### 3) TransformInterceptor ламає Swagger UI

Симптоми: Swagger UI не завантажується або показує помилку JSON.

#### Що зробити:

- Swagger UI обслуговується як статичні файли — Interceptor не повинен трансформувати їх
- Якщо проблема виникає, виключіть шлях /api/docs з TransformInterceptor перевіркою URL у intercept()

### 4) LoggingInterceptor показує невірний statusCode для помилок

Симптоми: помилки 404/400 логуються як 200.

**Що зробити:**

- Це відомий нюанс: Interceptor працює ДО Exception Filter. Якщо контролер кидає виключення, Interceptor не бачить фінальний statusCode
- Для точного логування помилок використовуйте catchError у pipe:  
`.pipe(tap(...), catchError(err => { logger.error(...); throw err; })))`

**5) "Cannot find module '@nestjs/swagger'"**

Симптоми: помилка при компіляції.

**Що зробити:**

- Перевірити встановлення: `docker compose run --rm app npm ls @nestjs/swagger`
- Якщо відсутній: `docker compose run --rm app npm install @nestjs/swagger`
- Перезібрати: `docker compose up --build`